BeSA: Benchmarks for Simulating Agent-based Models in Architecture Research

Amina Sokoli A. Giray Yağlıkçı
CISPA Helmholtz Center for Information Security

***Abstract*—Agent-based modeling (ABM) is a widely used simulation paradigm in biology, epidemiology, finance, and social sciences, with simulation tools scaling to over a billion interacting agents. Although ABM simulations are critical to scale and run at high performance in many diverse domains, unfortunately, no benchmark suite is available for designing architectures for these simulations. In this paper, we present, BeSA, the first ABM benchmark suite (work-in-progress) aiming to guide systems and microarchitecture research. BeSA covers two different use-cases (epidemiology and tumor growth), with a third under development (biofilm growth) whose agent counts range from fixed in size to dynamically growing. We apply SimPoint-based sampling to the epidemiology and tumor growth simulations, which reduces the number of executed instructions by $283{,}000\times$ and $2{,}500\times$, each. In both cases, execution concentrates in a few SimPoint clusters that cover at least 75% of execution. To understand the performance bottlenecks of these simulations, we started with the epidemiology use case, which we ran on a 16-vCPU Google Cloud C4 VM. We swept the agent count from 10 million to 1 billion and collected hardware performance counters. We observed IPC stayed around 0.40 and the top-down memory-bound fraction around 60% across runs, which is consistent with a memory-bound workload.**

I. Introduction

Agent-based modeling is a bottom-up approach to modeling complex systems that focuses on agents, their individual behavior, and the interactions between them. This allows the structure of a complicated system to be studied without needing computing answers analytically. ABM models a system as a population of simple, autonomous agents that interact over time, enabling sophisticated simulations whose macro-scale, emergent behavior can be studied and whose rules can be varied to see how agent-level decisions change simulation outcomes. It has been applied across diverse domains including biology, epidemiology, finance, and social sciences (e.g., [1, 2, 3, 4]).

At each timestep, every agent updates its state based on its interactions with other agents in close proximity (i.e., neighbors); we call this unit of work an *agent step*. For example, the state-of-the-art ABM simulation framework, BioDynaMo, implements an agent step as a short computation wrapped around an irregular, pointer-chasing walk over a spatially evolving neighbor structure. This access pattern resembles that of pull-direction graph analytics, with a low compute-to-memory ratio, irregular memory accesses, and bulk-synchronous execution. To make matters worse, the neighbor graph is rebuilt at every timestep as agents move. Because of this reconstruction, we argue that ABM is a *distinct* graph-type workload that deserves dedicated architectural study. However, the architecture research needs a benchmark suite to identify and address the performance bottleneck and energy costs. Established suites (e.g., SPEC [5], PARSEC [6], and GAP [7]) do *not* capture ABM's combination of dynamic spatial indexing and per-timestep neighbor reconstruction, and *no* benchmark suite for agent-based modeling exists for computer architecture simulations.

To this end, we propose BeSA, the first ABM benchmark suite for architecture research. BeSA is a work-in-progress and covers three use-cases at different scales. We show-case BeSA in a performance bottleneck analysis of epidemiology simulations and find them to be memory-bound (IPC $\sim$0.40, memory-bound fraction $\sim$60%) and limited in population size by memory capacity.

This paper makes the following two contributions.

- We propose BeSA, the first ABM benchmark suite for architecture research.
- We characterize epidemiology simulations with BeSA and identify their performance bottlenecks.

II. Design goals of the benchmark suite

A benchmark suite in this research area earns its value along two axes: representativeness and reusability. The suite is **representative** when its use cases differ in memory behavior. Although every BioDynaMo use case runs the same core agent step, use cases differ in how their populations evolve and how much work each step performs, so their memory behavior differs too. We measure this by their SimPoint phase structure. **Reusability** refers to whether a suite's workloads are portable across simulators. We have run a set of representative regions on gem5 that are designed to run in other functional and cycle-accurate simulators as well.

III. Methodology

Large-scale agent-based simulations take too long to simulate cycle-accurately in full, so we applied SimPoint-based sampled simulation [8] to both use cases in the suite. For epidemiology, we ran 200 million agents for 1300 iterations; for tumor growth, we started from BioDynaMo's default initial population of 2,401 agents which grew to over 130k over 800 iterations. In each case, we generated basic-block vectors with Valgrind over 100-million-instruction intervals and clustered them using a gcc-9-compatible fork of SimPoint 3.2, selecting $k = 12$ for epidemiology and $k = 10$ for tumor growth.

Figure 1 shows the resulting cluster weights. We observe that the execution concentrates in a few clusters in both cases: the largest cluster accounts for 50.4% of epidemiology execution and 42.5% of tumor growth execution, and the top three clusters cover roughly 75% and 90%, respectively.

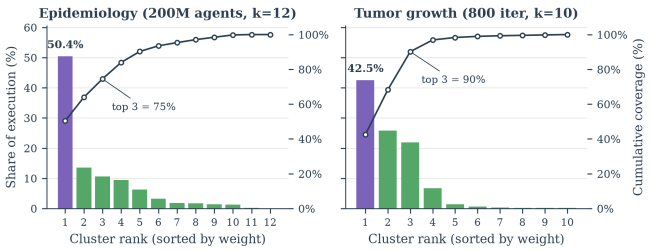


Figure 1: SimPoint cluster weights for the epidemiology and tumor growth use cases. In each plot, the bars show the share of execution in each SimPoint cluster (left axis), sorted by descending weight, and the overlaid curve shows the cumulative coverage of the ranked clusters (right axis). The highest weight cluster is colored in purple and the remaining clusters in green.

Table 1: SimPoint sampling parameters.

	Epidemiology	Tumor growth
Agents	200M	2,401 $\rightarrow$ 133,011*
Iterations	1300	800
BBV interval	100M insts	100M insts
Clustering	SimPoint 3.2 (gcc-9 fork)	
Clusters k	12	10
Regions simulated	12	10
Dyn. insts (total)	$\sim 3.4\times10^{14}$	$\sim 2.5\times10^{12}$
Insts simulated	1.2×10^{9}	1.0×10^{9}
Reduction	$\sim$283,000$\times$	$\sim$2,500$\times$

* Emerging population

IV. Case Study: Understanding the Performance Bottlenecks of Epidemiology Simulation

We analyze the performance bottlenecks of epidemiology simulations using BioDynaMo [9, 10] on Google Cloud C4 virtual machines (VMs), sweeping agent counts from 10 million to 1 billion with hardware performance unit (PMU) counters collected via *perf*. Table 2 lists the platform and workload configuration.

Table 2: Experimental setup for the characterization.

Platform	
VM	Google Cloud C4 (c4-standard-16)
CPU	Intel Xeon, Emerald Rapids
Cores	8 cores $\times$ 2 threads (16 vCPUs)
Memory	64 GB
OS	Ubuntu 22.04 LTS
Workload	
Agents	10M, 50M, 100M, 200M (225M, 1B attempted; OOM-killed)
Iterations	1300
Repetitions	3 per configuration

Figure 2 shows the microarchitectural characterization, and we make three observations from it. First, IPC stays within 0.40–0.45 across the entire sweep (Fig. 2(a)), so the core is underutilized at every scale we measured. Second, the memory-bound fraction from the top-down analysis stays around 60% at all completed runs (Fig. 2(b)). The 225M and 1B-agent runs did not complete due to memory capacity limitations. Third, per-agent memory footprint increases linearly. Peak memory follows $\text{mem}(N) = C + kN$, with fixed overhead $C \approx 1.59$ GB and marginal cost $k \approx 290$ bytes/agent ($R^2 = 0.9994$; Fig. 3).

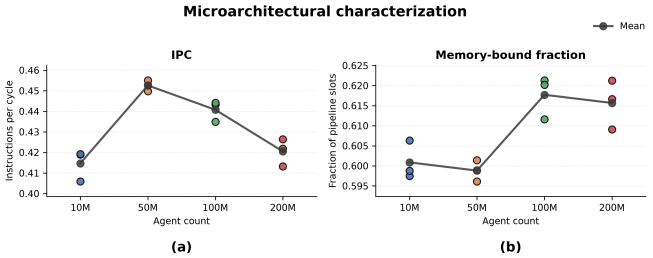


Figure 2: Microarchitectural characterization of BioDynaMo's epidemiology use case as a function of agent count (x-axis). (a) Instructions per cycle (IPC); (b) the memory-bount fraction of pipeline slots from the top-down analysis. In both plots, each colored point is one run and the line connects the per-count means over the three runs; Agent count is encoded by color: 10M blue, 50M orange, 100M green, 200M red.

The fit crosses the 64 GB VM limit near 221M agents, and the 225M and 1B-agent runs were both terminated by the Linux OOM killer, so **capacity** sets the ceiling on population size. (Fig. 3)

From these three observations, the first two are consistent with the memory-bound region of the DAMOV classification tree [11], while the third observation shows that memory capacity bounds the population a single node can simulate. Therefore, we conclude that the epidemiology use case is **memory-bound** and **capacity-limited**.

V. Ongoing and Future Work

Two threads of the work are currently in progress. First, we develop additional use cases to broaden the suite with cases such as *biofilm growth* [3]. Second, we implement a *cache-tiling* technique for the epidemiology use case. We hypothesize that the epidemiology use case is memory-bound because each timestep touches the entire agent population, whose working set exceeds the cache sizes. This causes the workload to access every agent at every iteration, so agent data is evicted and refetched from DRAM every iteration. To test this, we partition the population into spatially local tiles sized to fit in the cache. We expect to report initial tiling results at the workshop.

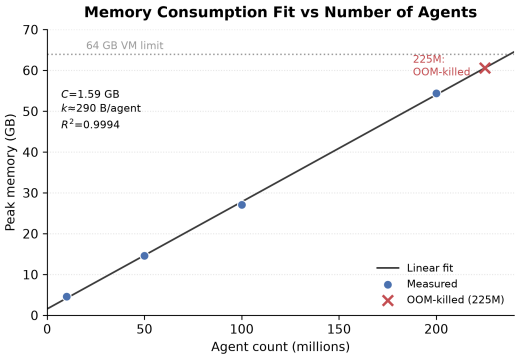


Figure 3: Memory consumption of the epidemiology use case versus agent count.

References

[1] Jean de Montigny et al. An in silico hybrid continuum-/agent-based procedure to modelling cancer development: Interrogating the interplay amongst glioma invasion, vascularity and necrosis. *Methods*, 185:94–104, 2021. Methods on simulation in biomedicine.

[2] Tobias Duswald et al. Bridging scales: A hybrid model to simulate vascular tumor growth and treatment response. *Computer Methods in Applied Mechanics and Engineering*, 418:116566, 2024.

[3] Ryan Bournes et al. Agent-based modeling of erosion and sloughing during growth of *Pseudomonas aeruginosa* biofilms. *npj Biofilms and Microbiomes*, June 2026. Advance online publication.

[4] Nature. The EU needs to back its ambition to end animal testing with cash. *Nature*, 654(8119):572, June 2026. Editorial.

[5] John L. Henning. SPEC CPU2006 benchmark descriptions. *ACM SIGARCH Computer Architecture News*, 34(4):1–17, 2006.

[6] Christian Bienia et al. The PARSEC benchmark suite: Characterization and architectural implications. In *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques*, PACT '08, pages 72–81. Association for Computing Machinery, 2008.

[7] Scott Beamer et al. The GAP benchmark suite. *arXiv preprint arXiv:1508.03619*, 2015.

[8] Greg Hamerly et al. SimPoint 3.0: Faster and more flexible program phase analysis. *Journal of Instruction-Level Parallelism*, 7:1–28, September 2005.

[9] Lukas Breitwieser et al. High-performance and scalable agent-based simulation with biodynamo. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, PPoPP '23, page 174–188, New York, NY, USA, 2023. Association for Computing Machinery.

[10] Lukas Breitwieser et al. BioDynaMo: a modular platform for high-performance agent-based simulation. *Bioinformatics*, 38(2):453–460, 09 2021.

[11] Geraldo F. Oliveira et al. DAMOV: A New Methodology and Benchmark Suite for Evaluating Data Movement Bottlenecks. *IEEE Access*, 9:134457–134502, 2021. Conference Name: IEEE Access.